

Guía Laborarotio

Objetivo

Familiarizarse con las herramientas fundamentales del lenguaje Assembly para la arquitectura x86-64.

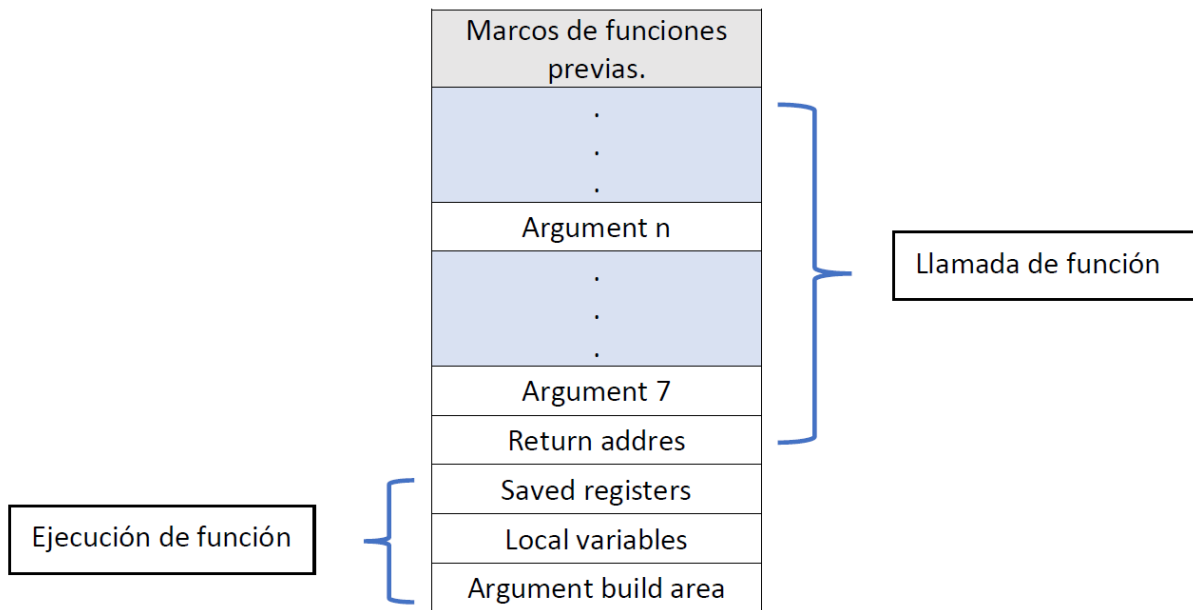
Registros

La arquitectura x86-64 cuenta con 16 registros de propósito general, cada uno de 64 bits, utilizados para almacenar datos, direcciones y controlar el flujo de ejecución.

64-bit	32-bit	16-bit	8-bit	Propósito principal
%rax	%eax	%ax	%al	Acumulador, valor de retorno, operaciones aritméticas.
%rbx	%ebx	%bx	%bl	Base. Preservado entre llamadas (callee-saved).
%rcx	%ecx	%cx	%cl	Contador, 4° argumento de función.
%rdx	%edx	%dx	%dl	Datos, 3° argumento de función.
%rsi	%esi	%si	%sil	Source, 2° argumento de función.
%rdi	%edi	%di	%dil	Destination, 1° argumento de función.
%rbp	%ebp	%bp	%bpl	Base Pointer, inicio del marco de pila.
%rsp	%esp	%sp	%spl	Stack Pointer, tope de la pila.
%r8				Registros extendidos, argumentos y uso general.
%r9				Registros extendidos, argumentos y uso general.
%r10				Registros extendidos, argumentos y uso general.
%r11				Registros extendidos, argumentos y uso general.
%r12				Registros extendidos, argumentos y uso general.
%r13				Registros extendidos, argumentos y uso general.
%r14				Registros extendidos, argumentos y uso general.
%r15				Registros extendidos, argumentos y uso general.

Pila

La pila organiza la información necesaria para la ejecución de funciones, guardando la dirección de retorno, registros, variables locales y argumentos adicionales. Crece hacia direcciones bajas.



Prologue y Epilogue

El **prologue** prepara el marco de pila, guardando el puntero base anterior y reservando espacio para variables locales. El **epilogue** restaura el puntero base y limpia la pila para devolver el control a la función llamante.

```
my_function:
    push rbp                ; Guarda el antiguo puntero de marco
    mov rsp, rbp            ; Establece el nuevo puntero de marco
    sub rsp, 16             ; Reserva espacio para variables locales
    Cuerpo de la funcion
    leave                   ; Equivale a mov rsp,rbp + pop rbp
    ret                     ; Regresa al llamador
```

```
.globl suma
.type suma, @function
suma:
push %rbp
mov %rsp, %rbp
mov %edi, %eax
add %esi, %eax
leave
ret
.globl main
.type main, @function
main:
push %rbp
mov %rsp, %rbp
sub $16, %rsp
movl $5, -4(%rbp)
movl $10, -8(%rbp)
movl -4(%rbp), %edi
movl -8(%rbp), %esi
call suma
movl %eax, -12(%rbp)
movl -12(%rbp), %eax
mov %eax, %esi
lea .LC0(%rip), %rdi
mov $0, %eax
call printf@PLT
mov $0, %eax
leave
ret
.section .note.GNU-stack,"",@progbits
```

Instrucciones x86-64

Categoría	Instrucción	Descripción
Movimiento de datos	mov src, dst lea addr, dst	Copia valor de src a dst. Carga dirección efectiva.
Aritmética	add, sub, imul, idiv	Operaciones básicas.
Lógica	and, or, xor, not	Operaciones bit a bit.
Comparación	cmp, test	Actualizan flags según resultado.
Salto	jmp, je, jne, jg, jl, ...	Salto condicionales.
Pila	push, pop, call, ret, leave	Manejo del marco de pila.

Laboratorio

Implementar un visitor `Gencode` para la gramática:

```
Program ::= StmtList
StmtList ::= Stmt ( ';' Stmt ) *
  Stmt ::= id '=' Exp | print('Exp')
  Exp ::= Term ( ( '+' | '-' ) Term ) *
  Term ::= Factor ( ( '*' | '/' ) Factor ) *
  Factor ::= id | Num | '(' Exp ')'
```

Ejemplos

```
print(4);
print(5)
```

```
.data
print_fmt: .string "%ld\n"
.text
.globl main
main:
  pushq %rbp
  movq %rsp, %rbp
  movq $4, %rax
  movq %rax, %rsi
  leaq print_fmt(%rip), %rdi
  movl $0, %eax
  call printf@PLT
  movq $5, %rax
  movq %rax, %rsi
  leaq print_fmt(%rip), %rdi
  movl $0, %eax
  call printf@PLT
  movl $0, %eax
  leave
  ret
.section .note.GNU-stack,"",@progbits
```

print(4+5)

```
.data
print_fmt: .string "%ld\n"
.text
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    movq $4, %rax
    pushq %rax
    movq $5, %rax
    movq %rax, %rcx
    popq %rax
    addq %rcx, %rax
    movq %rax, %rsi
    leaq print_fmt(%rip), %rdi
    movl $0, %eax
    call printf@PLT
    movl $0, %eax
    leave
    ret
.section .note.GNU-stack,"",@progbits
```

```
x = 5;  
y = 10;  
z = 20;  
print(x+y);  
print(x-z)
```

```
.data  
print_fmt: .string "%ld\n"  
.text  
.globl main  
main:  
    pushq %rbp  
    movq %rsp, %rbp  
    movq $5, %rax  
    movq %rax, -8(%rbp)  
    movq $10, %rax  
    movq %rax, -16(%rbp)  
    movq $20, %rax  
    movq %rax, -24(%rbp)  
    movq -8(%rbp), %rax  
    pushq %rax  
    movq -16(%rbp), %rax  
    movq %rax, %rcx  
    popq %rax  
    addq %rcx, %rax  
    movq %rax, %rsi  
    leaq print_fmt(%rip), %rdi  
    movl $0, %eax  
    call printf@PLT  
    movq -8(%rbp), %rax  
    pushq %rax  
    movq -16(%rbp), %rax  
    movq %rax, %rcx  
    popq %rax  
    addq %rcx, %rax  
    movq %rax, %rsi  
    leaq print_fmt(%rip), %rdi  
    movl $0, %eax  
    call printf@PLT  
    movq -8(%rbp), %rax  
    pushq %rax  
    movq -24(%rbp), %rax  
    movq %rax, %rcx  
    popq %rax  
    subq %rcx, %rax  
    movq %rax, %rsi  
    leaq print_fmt(%rip), %rdi  
    movl $0, %eax  
    call printf@PLT  
    movl $0, %eax  
    leave  
    ret  
.section .note.GNU-stack,"",@progbits
```

```
x = 120;  
y = 50;  
z = x/y;  
w = x-y+6;  
print(z);  
print(w)
```

```
.data  
print_fmt: .string "%ld\n"  
.text  
.globl main  
main:  
    pushq %rbp  
    movq %rsp, %rbp  
    movq $120, %rax  
    movq %rax, -8(%rbp)  
    movq $50, %rax  
    movq %rax, -16(%rbp)  
    movq -8(%rbp), %rax  
    pushq %rax  
    movq -16(%rbp), %rax  
    movq %rax, %rcx  
    popq %rax  
    cqto  
    idivq %rcx  
    movq %rax, -24(%rbp)  
    movq -8(%rbp), %rax  
    pushq %rax  
    movq -16(%rbp), %rax  
    movq %rax, %rcx  
    popq %rax  
    subq %rcx, %rax  
    pushq %rax  
    movq $6, %rax  
    movq %rax, %rcx  
    popq %rax  
    addq %rcx, %rax  
    movq %rax, -32(%rbp)  
    movq -24(%rbp), %rax  
    movq %rax, %rsi  
    leaq print_fmt(%rip), %rdi  
    movl $0, %eax  
    call printf@PLT  
    movq -32(%rbp), %rax  
    movq %rax, %rsi  
    leaq print_fmt(%rip), %rdi  
    movl $0, %eax  
    call printf@PLT  
    movl $0, %eax  
    leave  
    ret  
.section .note.GNU-stack,"",@progbits
```